

PCAD

Programmazione Concorrente

Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

PROCESSI E THREAD

Processi

- Un processo è un **programma** (di natura statica) in corso di **esecuzione** (di natura dinamica). La sua esecuzione necessita un ambiente :
 - spazio di indirizzamento dedicato
 - input e output (per es. gli standard)
- Più processi possono eseguirsi sulla stessa macchina in un modo quasi-simultaneo
 - Se il sistema rispetta il **tempo condiviso** o è **multi-task**
 - Ciò è valido anche se c'è un solo processore
 - Il sistema operativo è responsabile dell'allocazione delle risorse (memoria,tempo processore, input/output etc)
 - Un processo **non ha accesso** allo spazio di indirizzamento degli altri
 - Abbiamo l'**illusione del parallelismo**

Creare processi Unix

- In C, è possibile creare un nuovo processo grazie alla funzione fork:
 - **pid_t fork(void);**
- Questa funzione:
 - Crea un processo figlio
 - Restituisce un identificatore del processo PID
 - È possibile verificare se è nel processo padre o figlio grazie al PID restituito
 - fork restituisce 0 per il figlio
 - fork restituisce l'identificatore del figlio al padre
- Altre funzioni utili:
 - **pid_t getpid(void);** per avere il PID del processo corrente
 - **pid_t getppid(void);** per avere il PID del processo padre

Creare processi Unix (2)

- In pratica:
 - Tutto è nello stesso codice
 - Si testa il valore restituito da fork per sapere se ci si trova nel processo figlio o padre:
 - **if(fork()==0) {...}**
 - Questo permette di distinguere l'esecuzione del processo padre da quella del figlio
- Bisogna fare attenzione ai seguenti fatti:
 - Le variabili non sono condivise
 - Alla creazione del figlio, lo spazio di indirizzamento del padre viene copiato

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int id=getpid();
    printf("Sono il %d\n",id);
    printf("Faccio un fork\n");
    int idfork=fork();
    if(idfork==0){
        int idfiglio=getpid();
        int idpadre=getppid();
        printf("Sono %d figlio di %d\n",idfiglio,idpadre);
    } else {
        printf("Il mio figlio e %d\n",idfork);
        int stato;
        waitpid(idfork, &stato, 0);
    }
    return 0;
}
```

Le variabile non sono condivise

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int main(){
    int x=0;
    int pid = fork();
    if ( pid == 0 ) {
        x=25;
        printf("Valore di x per il figlio:%d\n",x);
        printf("Fine del processo figlio\n");
    }
    else {
        while(x!=25){
            sleep(2);
            printf("Valore di x per il padre %d\n",x);
        }
        printf("Fine del processo padre\n");
    }
    return 0;
}
```

Fare eseguire un command a un figlio

- Si può usare la funzione `execvp`:

```
int execvp(const char *file, char *const argv[]);
```

- L'array `argv` contiene
 - il nome della command nel primo campo
 - gli argomenti nei campi seguenti
 - **deve finire con NULL**
- non crea nessun nuovo processo: semplicemente lo spazio di indirizzamento viene rimpiazzato con quello del nuovo programma

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int id=getpid();
    printf("Sono il processore %d\n",id);
    printf("Faccio un fork\n");
    int idfork=fork();
    if(idfork==0){
        int idfiglio=getpid();
        int idpadre=getppid();
        printf("Sono %d figlio di %d\n",idfiglio,idpadre);
        char *args[]={ "ls", "-l", NULL};
        execvp(args[0],args);
        printf("Fine exec\n");
    } else {
        printf("Il mio figlio e %d\n",idfork);
        int stato;
        waitpid(idfork, &stato, 0);
        printf("Fine del figlio\n");
    }
    return 0;
}
```


Comunicazione fra processi

- Per comunicare tra processi, si possono usare dei 'tubi' (pipe)
- I tubi sono dei canali unidirezionali considerati come dei descrittori di file
- Un processo può scrivere a un'estremità e un altro può leggere dall'altra estremità



- Poiché sono considerati come descrittori di file, quando un processo crea un figlio, quest'ultimo recupera la tabella con i descrittori del processo padre e ha accesso al tubo.

Usare tubi

- La funzione

```
int pipe(int fds[2])
```

crea un tubo e mette le sue due estremità nell'array fds

- Si scrive su fds[1] e si legge su fds[0]
- Dopo il fork, **entrambi** processi possono scrivere e leggere, ma affinché funzioni correttamente, è necessario chiudere il "lato" non utilizzato.
- Ad esempio, se il processo padre scrive, deve chiudere fds[0] per primo, e se il processo figlio legge, deve chiudere fds[1].
- È necessario chiudere bene appena si è finito di usare il tubo.

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fds[2];
    pipe(fds);
    int idfork=fork();
    if(idfork==0){
        close(fds[1]);
        char buffer[100];
        read(fds[0],buffer,sizeof(buffer));
        printf("Sono il figlio e ho ricevuto %s dal mio padre\n",buffer);
        close(fds[0]);
    } else {
        close(fds[0]);
        write(fds[1],"Luke, I am your father",22);
        int stato;
        waitpid(idfork, &stato, 0);
        close(fds[1]);
        return 0;
    }
}
```

Condividere delle variabile



- Useremmo dei processi leggeri (**thread**)

Threads vs Processi

- **Un thread** è un filo d'esecuzione all'interno di un programma.
- Il thread è eseguito da un processo
- Un processo può gestire più thread
 - Si parla di processo **multi-thread**
 - Almeno un thread è presente.
- Ogni filo d'esecuzione è diverso e presenta i seguenti attributi:
 - Un **puntatore di esecuzione** o **PC (Program Counter)**
 - Una stack d'esecuzione (**stack**)
- È più difficile programmare con i thread in C MA :
 - l'implementazione è più efficace
 - Condividere dei dati è più facile (ma è anche per questo che bisogna fare attenzione)

L'API POSIX

- In C, esiste una libreria 'classica' per la creazione e la manipolazione di thread
 - la libreria **POSIX**
- Per usarla, bisogna includerla nel codice:
 - **#include <pthread.h>**
- Bisogna anche specificare al momento della compilazione che si desidera utilizzare questa libreria con l'opzione -pthread o -lpthread
- Ad esempio, per compilare il file test.c:
 - **gcc -pthread -Wall test.c -o test**

Creazione di thread

- Per creare uno thread, si usa la funzione:

```
int pthread_create(  
    pthread_t *thread, //Per ricordare i dati del nuovo thread  
    const pthread_attr_t *attr, // Attributi dello thread  
    void *(*start_routine) (void *), // Funzione da eseguire  
    void *arg); // Argomenti della funzione
```

- In pratica, per gli attributi, si può mettere **NULL** per utilizzare gli attributi predefiniti
- Questi attributi permettono, per esempio, di definire una politica di ordinamento per lo scheduler
- Il codice eseguito dallo thread è dato dalla funzione **start_routine**
- Appena la funzione **pthread_create** termina, lo thread è creato e può iniziare l'esecuzione

Creazione di thread in pratica

- Si scrive una o più funzioni contenenti il codice da eseguire dai thread. Il prototipo deve essere:

```
void *function(void *)
```

- Per passare degli argomenti alla funzione al momento della creazione dello thread, si inseriscono come ultimo argomento della funzione `pthread_create`
- A volte è necessario attendere la fine dell'esecuzione degli thread creati.
 - se il programma principale termina prima, allora gli thread vengono distrutti
 - per questo scopo si può usare la funzione

```
int pthread_join(pthread_t thread, void **value_ptr)
```

- Il primo argomento corrisponde a quello creato da `pthread_create`
- `value_ptr` è il valore ritornato dallo thread chiamando la funzione

```
void pthread_exit(void *value_ptr)
```

- Attenzione: chiamare questa ultima funzione provoca la terminazione dello thread!!!

Esempio

- Bisogna a dare il codice degli thread

```
void *stampa(void *ptr){  
    char *s=(char *)ptr;  
    printf("Il mio messaggio e: %s",s);  
    return NULL;  
}
```

- Per creare un thread che esegue questa funzione:

```
pthread_t th1;  
char *s1="Sono il thread 1\n";  
int r1=pthread_create(&th1,NULL,stampa,s1);
```

- Alla fine, si può usare join per attendere la fine del thread

```
pthread_join(th1,NULL);
```

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void *stampa(void *ptr);

int main(){
    pthread_t th1, th2;
    char *s1="Sono il thread 1\n";
    char *s2="Sono il thread 2\n";
    pthread_create(&th1,NULL,stampa,s1);
    pthread_create(&th2,NULL,stampa,s2);

    pthread_join(th1,NULL);
    pthread_join(th2,NULL);
    return 0;
}

void *stampa(void *ptr){
    char *s=(char *)ptr;
    printf("Il mio messaggio e: %s",s);
    return NULL;
}
```

Esempio con valore di ritorno

```
int main(){
    pthread_t th1, th2;
    char *s1="Sono il thread 1\n";
    char *s2="Sono il thread 2\n";
    pthread_create(&th1,NULL,stampo,s1);
    pthread_create(&th2,NULL,stampo,s2);

    char *r1;
    char *r2;
    pthread_join(th1,(void **)&r1);
    pthread_join(th2,(void **)&r2);
    printf("%s",r1);
    printf("%s",r2);
    return 0;
}

void *stampo(void *ptr){
    char *s=(char *)ptr;
    printf("Il mio messaggio e: %s",s);
    char *mess=(char *)malloc(100*sizeof(char));
    strcat(mess,s);
    strcat(mess," Fini\n");
    return mess;
}
```

Esempio con pthread_exit

```
int main(){
    pthread_t th1, th2;
    char *s1="Sono il thread 1\n";
    char *s2="Sono il thread 2\n";
    pthread_create(&th1,NULL, stampa, s1);
    pthread_create(&th2,NULL, stampa, s2);

    char *r1;
    char *r2;
    pthread_join(th1, (void **)&r1);
    pthread_join(th2, (void **)&r2);
    printf("%s", r1);
    printf("%s", r2);
    return 0;
}

void *stampa(void *ptr){
    char *s=(char *)ptr;
    printf("Il mio messaggio e: %s", s);
    char *mess=(char *)malloc(100*sizeof(char));
    strcat(mess, s);
    strcat(mess, " Fini\n");
    pthread_exit(mess);
}
```

Condividere delle variabile



- Si può usare una variabile globale
- o passare il loro indirizzo come argomento di **pthread_create**
- Le variabili devono essere dichiarate come **volatile**

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <unistd.h>

void *inc(void *ptr);

volatile int a=0;

int main(){
    pthread_t th1, th2;
    pthread_create(&th1, NULL, inc, NULL);
    pthread_create(&th2, NULL, inc, NULL);

    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    printf("a vale %d\n", a);
    return 0;
}

void *inc(void *ptr){
    a=a+1;
    return NULL;
}
```

Attenzione con i dati condivisi

- Gli thread si eseguono in parallelo
- Non è possibile fare ipotesi sullo scheduler (è lui che interrompe gli thread e che decide quale thread deve essere eseguito in ogni momento)
- Se non si presta attenzione alla manipolazione delle variabili condivise, si possono osservare dei comportamenti strani.
- Ad esempio:
 - un thread che testa il valore di una variabile intera prima di ridurla
 - viene interrotto fra il test e l'update
 - un altro thread imposta il valore della variabile a 0
 - il primo thread imposta la variabile a -1, mentre forse non era quello che si voleva ottenere

Esempio di dati condivisi

```
volatile int a=0;

int main(){
    pthread_t th1, th2, th3;
    pthread_create(&th1, NULL, inc, NULL);
    pthread_create(&th2, NULL, dec, NULL);
    pthread_create(&th3, NULL, dec, NULL);
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    pthread_join(th3, NULL);
    printf("a vale %d\n", a);
    return 0;
}

void *inc(void *ptr){
    a=a+1;
    return NULL;
}

void *dec(void *ptr){
    if(a>0){
        a=a-1;
    }
    return NULL;
}
```

**Questo codice
potrebbe
stampare -1 !!!**